

TECHNICAL DOCUMENTATION

Project 3: Content-Based Filtering & Match-Score Ranking Engine

Intern Name:	Sake Nishanth
Domain:	Artificial Intelligence
Organization:	DecodeLabs
Algorithm:	Set-Theoretic Content Recommendation

1. Introduction & Concept Framework

Modern personalization engines rely on data relevance scoring to filter through extensive item metadata catalogs. This project details the construction of a **Content-Based Filtering Recommendation System** engineered using pure Python structures. The application takes a variable list of user preferences and matches them against an indexed educational course dictionary containing descriptive feature tokens.

2. Mathematical Matching Engine Logic

The matching system avoids vector scaling dependencies, utilizing discrete set intersection metrics instead. Let the user preference tokens be a set P , derived by parsing the raw comma-separated string input. Let the catalog contain a finite series of items, where each item I_k is mapped to a set of pre-assigned domain metadata tags T_k .

The system calculates the similarity match metric $S(P, T_k)$ using the intersection cardinality:

$$S(P, T_k) = |P \cap T_k|$$

Following the scoring iteration, a filtering constraint is applied to isolate recommendations:

$$R = \{(I_k, S(P, T_k)) \mid S(P, T_k) > 0\}$$

The resulting array R is ordered using a descending sort function based on the match scores, ensuring the most relevant items are prioritized at the top of the recommendation queue.

3. Algorithmic Source Code Structure

```
# Core Intersection Matching Engine
user_input = input("Enter your interests (comma separated): ").lower()
user_preferences = [interest.strip() for interest in user_input.split(",")]
```

```
recommendations = []

for item, tags in items.items():
    similarity_score = len(set(user_preferences) & set(tags))
    if similarity_score > 0:
        recommendations.append((item, similarity_score))

recommendations.sort(key=lambda x: x[1], reverse=True)
```

4. Empirical Verification & Interface Logs

The application was tested using a multi-token query sequence: "python,html,ai,nlp,aws,data". The system accurately calculated the overlapping feature metrics and ordered the output list dynamically.

```
# AI Recommendation Logic

items = {
    "Python Course": ["python", "programming"],
    "Machine Learning Course": ["python", "ai", "machine learning"],
    "Data Science Course": ["python", "data science", "analytics"],
    "Web Development Course": ["html", "css", "javascript"],
    "Deep Learning Course": ["ai", "machine learning", "python"],
    "Computer Vision Course": ["ai", "python", "computer vision"],
    "Natural Language Processing": ["ai", "nlp", "python"],
    "Generative AI Fundamentals": ["ai", "llm", "machine learning"],
    "Cloud Computing": ["aws", "cloud", "devops"],
    "Cyber Security Basics": ["security", "networking", "ethical hacking"],
    "Database Management": ["sql", "database", "data"],
    "Java Programming": ["java", "programming"],
    "C++ Programming": ["c++", "programming"],
}
```

```
==== AI Recommendation System ====
Enter your interests (comma separated): python,html,ai,nlp,aws,data

Recommended Items:
- Natural Language Processing (Match Score: 3)
- Machine Learning Course (Match Score: 2)
- Deep Learning Course (Match Score: 2)
- Computer Vision Course (Match Score: 2)
- Python Course (Match Score: 1)
- Data Science Course (Match Score: 1)
- Web Development Course (Match Score: 1)
- Generative AI Fundamentals (Match Score: 1)
- Cloud Computing (Match Score: 1)
- Database Management (Match Score: 1)
- Data Visualization (Match Score: 1)
```

Figure 3.1: Recommendation output showing dynamic match scoring, set intersecting, and descending rank order.

As shown in Figure 3.1, *Natural Language Processing* achieves the highest score of 3 due to exact intersections across the 'ai', 'nlp', and 'python' tags, while related courses scale downwards predictably.